E1 研究思路：公平比较、复杂度与结论边界

Oct 20, 2026 · @geniussuzane

研究问题是怎么形成的

最初，我想预测哪些订单容易延误。做下去之后我发现，“延误”是实际配送天数和预计天数之间的比较。因此，它不仅取决于配送快慢，也取决于预计日期设得多宽松。于是问题变成：更复杂的预测方法，能否在保持可靠性的同时给出更紧凑的预计日期？

这个问题不是一开始就设计好的。它是在做项目、接受审阅和反复讨论中逐步形成的。下面的三个问题也是这样：有些规则是在发现旧比较有漏洞之后才补上的。我把这个过程如实写出来。

问题一：如何公平比较

公平比较的意思是：两种方法只在“方法本身”上不同，其余条件都相同。旧的第 7 阶段比较没有做到这一点。按州规则多用了 17,056 张更新、延误率更高（8.74%）的第四季度订单；回归只用更早的拟合期，再靠平移补偿。因此，旧的“回归 vs 州规则”差异里混入了数据新鲜度。这个问题是在审阅中发现的，E1 把它拆开了。

条件	旧比较（第 7 阶段）	E1
拟合样本	州规则：F + C；回归：F	两者都只用 F（25,666 单）
标签可用时间	冻结时点 2018-04-01	每个决策时点只用此前已签收的订单：选设定用 2017-12-01，拟合与校准用 2018-04-01
卖家历史快照	拟合行按月更新，校准行和评价行冻结在窗口起点	所有行统一用下单月月初已知的历史（规则 U）；旧口径 O 保留做对照
校准	只有回归平移	所有策略用同一个校准集 C（17,056 单）和同一个平移函数
评价	E（19,950 单）	同一批 E，冻结之后只评一次

只统一条件还不够。我还想知道每个条件各自改变了多少。因此我从旧比较（R0）出发，每步只改一件事：R1 改州规则的拟合样本，R2 给州规则加上同样的校准，R3 改快照口径。预先规定主判断只看 R3 对 R0，中间步骤只作描述。这样不会在看到结果后挑一个最有利的比较。需要注意：这是按固定顺序的对照，不是唯一的原因分解。

问题二：复杂度是否有用

我在拟合任何新模型之前，就把“什么算值得改”写进了登记配置（v3，哈希 ae6a4a46…）。这样，“不值得改”是一个事先允许的结果，而不是失败。

判断	门槛	用在哪里
线性模型的每一步改动	V 上 pinball 至少下降 1% 才采纳，否则保留更简单的前一步	选设定（F0 → V）
近似平局	损失 ≤ 1.001 × 全局最小值的候选算平局，取更简单的（正则更强、树更浅、迭代更少）	选设定
H2：线性改动是否有实际意义	相对改善 g ≥ 2%，且配对区间上端 < 0	最终评价
H3 / 是否替换现有规则	平均预计天数至少短 1 天，覆盖率最多低 1 个百分点，且没有 ≥ 1,000 单的月份低于目标 3 个点以上	最终评价，分别对公平参照和现有规则报告

这些门槛是实际意义标准，不是统计显著性。比如 1% 的意思是：比这更小的改善，不值得让模型变复杂。pinball 更低也不足以让一个策略胜出，因为顾客看到的是日期，不是损失。另外，即使 HGB 在 V 上不如线性，它也会在 E 上评价一次。这是为了避免只报告好结果。

这个设计也有代价：逐步搜索可能漏掉“两个改动一起才有效”的组合。因此它是一个有限的搜索，不是“最优模型”的证明。

问题三：结果能说明什么

同一个实验可以回答三个不同层次的问题。它们容易被混在一起，所以我分开写。

层次	问的是什么	指标	本项目能否回答
预测误差	预测器是否更接近真实的 90% 分位数	pinball loss	能
日期兑现	变成整数日期并校准后，日期是否更紧、是否还可靠	覆盖率、平均预计天数、未覆盖订单的超出天数、月度覆盖率	能，但只对这段评价期
实际配送改善	包裹是否送得更快、顾客是否更满意	配送天数、评分	不能

第一层和第二层可能不一致。pinball 更低，取整和平移后日期却可能没有短到 1 天。因此替换判断用第二层。

第三层本项目回答不了。预计日期不改变物流，包裹不会因为日期写得紧就送得快。而且我只有历史观测数据，没有实验。更紧的日期对顾客评分、下单意愿的影响，需要 A/B 测试之类的设计才能估计。所以我不会把“日期更紧”写成“配送改善了”。

为什么不用平均配送时间设定预计日期

结论：如果晚到和余量的单位惩罚不同，最优预计天数是一个分位数，不是平均值。在 V 的 12,045 单上，按平均值设日期只有 65.5% 的订单按时到达。

设实际配送天数为 Y，给出的预计天数为 d。考虑一个简化损失：第一项惩罚晚到的天数，第二项惩罚给得过宽的余量。

$$L(d, Y) = c_{late} \cdot (Y - d)_+ + c_{slack} \cdot (d - Y)_+$$

连续情形。 记 R(d) = E[L(d,Y)]，F 为 Y 的分布函数。对 d 求导并令其为 0：

$$R'(d) = -c_{late} \cdot P(Y > d) + c_{slack} \cdot P(Y \le d) = (c_{late} + c_{slack}) \cdot F(d) - c_{late} = 0$$，因此 $$F(d^\star) = q = \frac{c_{late}}{c_{late} + c_{slack}}$$

整数天。 配送天数和预计天数都是整数，所以我直接看把 d 加 1 天的变化。Y > d 的订单少晚一天，各省 c_late；Y ⩽ d 的订单多一天余量，各增加 c_slack。因此：

$$R(d+1) - R(d) = (c_{late} + c_{slack}) \cdot F(d) - c_{late}$$

这个差随 d 增加而不减。所以最优的 d* 是满足 F(d) ⩾ q 的最小整数。这正好是项目里州规则用的经验分位数定义（numpy 的 inverted_cdf）。另外，分位数 q 上的 pinball loss 就是 c_late = q、c_slack = 1 − q 的这个损失（scikit-learn 说明）。因此，90% 分位数对应 9:1 的单位惩罚比。

平均值对应的是平方损失，中位数对应 1:1 的绝对损失。两者都把晚到和提前当作同样的代价。配送天数右偏时，平均值离 90% 分位数很远。

用 V 数据验证。 下表把一个统一的天数给所有订单（不用任何特征），在 9:1 损失下比较。我在 d = 1…60 中逐个计算，损失最小的是 d = 20，恰好是经验 90% 分位数：F(19) = 89.6% < 90% ⩽ F(20) = 91.4%。

预计天数的取法	d（天）	按时到达比例	9:1 损失（每单）
平均值 11.43，向上取整	12	65.5%	23.76
中位数	10	53.1%	30.51
经验 90% 分位数	20	91.4%	15.61

模型的做法是让 d 随订单特征 x 变化，同一个推导对条件分布 F(· | x) 成立。这是事后的描述性计算，只用 V，不影响任何选择（`scripts/e1_posthoc_v.py`）。

这个模型的局限。

- 90% 分位数在这个简化模型里对应 9:1 的单位惩罚比。这是选择 q = 0.90 的推论，不是我测量出来的。
- 这不代表 Olist 的真实商业成本就是 9:1。我没有成本数据。
- 顾客损失未必随晚到天数线性增长。可能“晚到一天”就已经造成大部分代价；那时更接近一个覆盖率约束，而不是线性成本。
- 不同顾客、不同商品的成本比也可能不同。
- 推导假设我知道真实分布。实际上分布是估计的，而且可能随时间变化（见下面的小实验）。

因此，项目仍然展示覆盖率与预计天数的取舍，不声称找到了利润最优的日期。

把“不改线性模型”写成有证据的结果

结论：在预先限定的逐步搜索中，星期编码、目标尺度和正则化的候选改动，都没有达到本项目设定的采纳门槛。因此保留原线性设定（C* = C0）。我不把它写成“正则化无效”或“类别编码没用”。

候选	V 上的 pinball	相对 C0 的改善	决定
C0 注册设定	1.28856	—	起点
C1 星期几改类别编码	1.29170	−0.24%	拒绝
C2 目标改原始天数	1.28932	−0.06%	拒绝
C3 L1 正则 α = 0.0001（3 个 α 中最好的）	1.28713	+0.11%	拒绝（< 1%）

为什么 0.111% 的改善不值得自动采纳？

1. 它低于事先写好的 1% 门槛。门槛是在看到结果前定的，所以我不能因为“差一点”而放宽。
2. 它的绝对大小是每单 0.0014 个损失单位，却要多一个需要调的参数。
3. 我事后在 V 上按下单周做了配对自助法（14 周，2,000 次）。改善的 95% 区间是 −0.002% 到 0.245%。即使按乐观的一端，也远低于 1%。事后区间没有显示达到预设 1% 门槛的改善，因此不改变原选择。候选本来就是在同一个 V 上选出的，这个区间也不包含选择和重新拟合带来的不确定性；逐单文件、重抽样周索引和计算公式见 outputs/e1/posthoc/。

为什么这不等于证明没有改善空间？

- 逐步搜索只试了少数几个改动，可能漏掉“一起改才有效”的组合，也没有试新特征。
- V 只是 2017 年 7–9 月，延误率 3.4%。校准期是 8.74%，评价期是 4.26%。在其他时期，结论可能不同。

- 门槛本身是一个判断，不是自然规律。

D3 的优势只是进入下一步的理由。 浅层 HGB（深度 2、400 次迭代）在 V 上的损失比 C0 低 2.01%（事后区间 1.11% 到 2.85%）。但它是 8 个候选里在 V 上最好的，所以这个数字偏乐观。而且这是未校准的预测误差，不是日期兑现。它不是 H2 的证据（H2 比较的是线性改动），也不是 H3 的证据（H3 看校准后的天数、覆盖率和月度可靠性）。我不会把它写成“优化了交付策略”。

还有一个后果：因为 C* = C0，C* 在 F 上重拟合后与 B3 是同一个模型。H2 会记为“未选中线性改动”，我不会把两个名字当作两份独立证据。

小实验：校准失效来自时间变化，还是只看到快速签收的订单？

结论：两种机制都会让覆盖率偏离目标，但痕迹不同。只用已签收订单时，平移量本身变小，预计天数和覆盖率一起下降。配送分布随时间变化时，平移量和预计天数不变，只有覆盖率移动。

设定。 这是合成数据的补充实验，不属于 E1 的登记运行，也不碰 Olist 数据。配送天数按 log Y = 2.2 + 0.5x + 0.45ε 生成（x ~ U(0,1)，ε ~ N(0,1)），再向上取整。预测器固定为条件中位数，故意不是 90% 分位数，这样校准才有作用。校准用项目里同一个平移函数（q = 0.90）。每次重复重新抽 2,000 单校准、 5,000 单评价，共重复 1,000 次（第一轮，种子 20261020）。

1. **稳定、标签完整：** 校准和评价来自同一分布。
2. **稳定、只用已签收订单：** 校准订单在 90 天窗口内均匀下单，冻结在窗口末；只有下单时间 + 配送天数 < 90 的订单可用。
3. **标签完整、之后配送变化：** 评价期在取整之前把连续配送时长乘以 1.15（变慢）或 0.85（变快），再向上取整。

场景	可用校准单数	平移量（天）	平均预计天数	覆盖率（5%–95%）	低于 90% 的重复占比
1 稳定、标签完整	2,000	9.64	21.85	91.7%（90.6–92.8）	1%
2 稳定、只用已签收	1,701	8.64	20.85	90.0%（88.7–91.3）	47%
3a 之后变慢 15%	2,000	9.64	21.85	85.9%（84.3–87.5）	100%
3b 之后变快 15%	2,000	9.64	21.85	95.9%（95.2–96.6）	0%

括号内是第 5–95 百分位数，即中间 90% 的重复落在的范围，不是均值的置信区间。场景 1 的覆盖率略高于 90%，是因为天数是整数、预计日期向上取整。场景 2 的偏差大小取决于有多少订单还在路上。把窗口缩短到 45 天（约 70% 可用），覆盖率降到 87.1%；30 天（约 56% 可用）降到 82.2%；180 天（约 93% 可用）为 91.0%。作为参照，Olist 的 2018 年第一季度在冻结时约 81% 已签收。

配对敏感性运行（第二轮，新增；上表是第一轮，保留不动）。 第一轮里，场景 1 和 2 共用同一批校准订单，但每个场景各自抽评价订单。第二轮在每次重复中让所有场景共用评价订单；场景 3 只对同一批抽样缩放配送时长。另加一个诊断：从完整队列里随机抽与已签收子集同样数量的订单。下表是与场景 1 的配对差（1,000 次重复，种子 20261021；均值的 Monte Carlo 标准误与第 5–95 百分位数分开报告）。

与场景 1 的配对差	覆盖率差（点）均值 ± MC 标准误	覆盖率差 5%–95%	预计天数差均值
2 只用已签收订单	−1.68 ± 0.015	−2.48 到 −0.94	−0.99
诊断：同样数量的随机子集	+0.02 ± 0.009	−0.42 到 +0.46	+0.02
3a 之后变慢 15%	−5.77 ± 0.014	−6.50 到 −5.08	0
3b 之后变快 15%	+4.26 ± 0.012	+3.66 到 +4.90	0

随机子集几乎不改变覆盖率，而同样数量的已签收子集使覆盖率下降约 1.7 点。因此，在这个设定里，偏差来自“只看到快速签收”的选择性缺失，而不是标签变少。第二轮的结论与第一轮一致。两轮的逐重复结果、生成代码和哈希都已保存；`scripts/e1_verify_supplementary.py` 从保存文件独立重算，43 项全部一致。

这个实验能帮什么。 它给出一个可检查的区分方法。等一个校准窗口的标签全部到齐后，重算一次平移量，和冻结时算的比较：差距反映“只看到快速签收”的偏差。而在预计日期不变的情况下覆盖率漂移，更像是分布随时间变化。

它不能证明什么。 它只说明两种机制在这个合成设定下的样子，不证明 Olist 真实数据的问题一定来自其中一种。两者也可能同时发生。另外，普通 split conformal 的覆盖保证依赖校准样本与新样本可交换。场景 2 和场景 3 正好是不可交换的情形，所以不能直接照搬那个保证（Barber, Candès, Ramdas & Tibshirani, *Conformal prediction beyond exchangeability*）。

代码与结果：`scripts/e1_calibration_sim.py`，`outputs/e1/sim/`（种子 20261020，可复现）。

E1 最终结果（冻结后在 E 上只评一次）

结论：公平比较之后，线性回归对州规则的判断反转了；但更复杂的 HGB 没有带来足够的日期改善；主校准比较里的策略（同一个 C、同一个平移接口）都没有达到替换项目保留的旧州规则的标准。E 共 19,950 单、17 个下单周；区间为按周自助法的 95% 百分位数区间。

H1：统一条件后，结论变了。 差值为“回归策略 − 州规则策略”。

步骤	州规则（天 / 覆盖率）	回归（天 / 覆盖率）	天数差	覆盖率差	1 天 / 1 点判定
R0 旧比较	20.31 / 93.95%	22.87 / 96.45%	+2.57	+2.50 点	不采用回归
R1 州规则只用 F	17.99 / 90.75%	22.87 / 96.45%	+4.88	+5.69 点	不采用
R2 州规则加同样的 C 平移	23.99 / 96.49%	22.87 / 96.45%	−1.12	−0.04 点	采用
R3 再改 U 快照	23.99 / 96.49%	22.45 / 96.18%	−1.54	−0.31 点	采用

R3 对 R0 的天数差变化 −4.11 天，判定也改变，按预先规则达到 H1 标准。R3 的天数差区间是 −1.78 到 −1.30 天，覆盖率差是 −0.63 到 −0.01 点。按固定顺序看，最大的变化发生在 R2：州规则用上同一个校准集后，需要加 6 天。因此，在预设的比较顺序下，仅改变设计条件就足以改变相对判断。这是设计敏感性，不是对方法优劣的因果识别。

问题	比较	结果	判定
H2 线性改动	C* 对 B3	C* = C0，与 B3 是同一个模型；g = 0 是构造结果，不是两个模型互相验证	未选中线性改动，不支持
H3 浅层 HGB	D* 对公平参照 B1c	短 0.32 天（−0.55 到 −0.17），覆盖率高 0.61 点	不到 1 天，不支持
是否替换现有规则	D* 、C* 对旧州规则 B0（20.31 天 / 93.95%）	D* 长 3.37 天，C* 长 2.15 天	都不替换
与 Olist 对照（q = 0.9126）	C* 对 Olist 日期（24.44 天 / 95.74%）	短 1.02 天，覆盖率高 0.88 点	以很小的余量通过规则

预测损失更低，不等于日期更短。 HGB 的未校准整数日 pinball loss 比线性低约 6.5%（1.324 对 1.417），但它的未取整预测均值本身就高约 0.90 天。校准平移再增加约 0.31 天的相对差距，取整及下限处理贡献约 0.006 天，最终平均日期高约 1.22 天。

构成（天）	线性 C*	HGB D*	D* − C*
未取整预测均值	15.809	16.709	+0.900
C 上的平移量	6.148	6.459	+0.311
取整及下限处理	0.497	0.503	+0.006
最终平均预计天数（各值按完整精度直接舍入，分项未必恰好相加）	22.453	23.671	+1.217

这说明降低分位数损失与缩短平均预计日期是不同的目标，不能只凭损失改善决定替换策略。一个帮助理解的数字：未校准的整数日期覆盖率，线性为 88.33%，HGB 为 91.08%。pinball 对晚到的惩罚是余量的 9 倍，所以更长、更少晚到的日期可以有更低的损失。这不是建议事后把未校准的 HGB 选为新赢家。

主校准策略在 E 上明显过覆盖。 采用同一个 C 和同一个平移接口的策略（B1c、B2、B3、C*、D*），目标是 90%，实际为 96%–97%，每个月都高于目标。未校准对照（B1、旧 B0）保留，不在此列。在这段评价期，统一校准后的策略明显超过目标覆盖率，却没有满足替换旧 B0 的日期缩短标准。

这与校准分数随时间变化的解释相容，但本研究尚未区分配送时长、原预计日期和订单构成变化的作用。C 期按原日期计的延误率是 8.74%，E 期是 4.26%，这个差异本身同时受三者影响，不能直接推出配送变快。另外，旧 B0 用 F + C 拟合且不平移，与新策略的差别同时涉及训练信息的使用方式和校准。因此，结论是不替换本轮策略，而不是模型或校准方法普遍无效。校准方式是值得继续研究的环节，但尚未证明它是主要原因。

与 Olist 的对照是 q = 0.9126 的结果，要与上面 q = 0.90 的结果分开看。 C* 只比 Olist 日期短 1.02 天，刚好越过 1 天的门槛。我事后用同一组重抽样周算了区间：天数差为 −3.25 到 +0.69 天，包含 0。因此它不能支持已确认的日期优势，我不把它写成“比 Olist 更紧且同样可靠”。

证据：冻结记录 `outputs/e1/freeze/`；评价输出和逐单文件 `outputs/e1/score/`。`scripts/e1_verify_score.py` 只用 numpy / pandas 从保存的文件重算所有数字，43 项全部一致。

目前能说与不能说

截至 2026-10-20，E1 已按登记顺序完成：F0 / V 选设定、F 上拟合、C 上校准、冻结，然后在 E 上评价一次。但 E 这段订单在第 7 阶段已经分析过，“冻结后只评一次”不等于盲测，E1 仍是有事前登记的探索性扩展。

可以说	还不能说
在预设的比较顺序下，仅改变设计条件就改变了回归与州规则的相对判断（R3：短约 1.5 天，覆盖率低 0.3 点）	这是方法优劣的因果识别；回归在其他时期也优于州规则
本次有限的搜索没有选中线性改动	正则化、类别编码“没用”
浅层 HGB 预测损失更低，但日期更长；差距主要在校准前就已存在	HGB 给出了更紧且同样可靠的日期
主校准策略都没有达到替换旧 B0 的标准，且在 E 上明显过覆盖	新方法“没有价值”；劣势主要来自校准；配送变快了

可以说	还不能说
q = 0.9126 时，C* 按预设规则比 Olist 日期短 1.02 天	比 Olist 更紧且同样可靠（事后区间包含 0）
90% 分位数在线性损失下对应 9:1 的惩罚比	Olist 的真实成本比是 9:1，或找到了利润最优日期
模拟中，“只看到快速签收”和“分布变化”留下不同痕迹	Olist 的过覆盖由其中某一种导致
预计日期可以在覆盖率与天数之间取舍	更紧的日期让配送变快或顾客更满意

写作与 AI 协助说明。 本页由 AI 助手协助起草，推导、E1 的实验代码（选设定、冻结、评分）、模拟和事后计算也由 AI 协助完成，并经过多轮独立审阅复核。本页很多解读上的收紧（如 HGB 日期差的分解、过覆盖原因的写法）来自审阅意见。研究问题、三个问题的框架和小实验的场景设计来自项目讨论。第一人称的句子是草稿，需要作者核对并改成自己的话。正式使用前，作者应能独立重做整数天的推导，并解释模拟中场景 2 的删失规则。